

Machine Learning Fundamentals

Week 3 – Task 1

Model Evaluation & Hyperparameter Tuning

Prepared By: Abdul Raheem

Course: Machine Learning Fundamentals

Organization: Neurofive Solutions

Dataset: Titanic – Machine Learning from Disaster

Problem Type

Binary Classification

This notebook evaluates the Logistic Regression model developed in Week 2 using advanced evaluation metrics and improves its performance through hyperparameter tuning.

Objectives

The objectives of this notebook are:

- Reload the Titanic dataset
- Prepare the dataset for classification
- Train a baseline Logistic Regression model
- Evaluate the model using Accuracy, Precision, Recall, and F1-Score
- Explain why accuracy alone is not always sufficient
- Tune the model using GridSearchCV
- Compare original and tuned model performance
- Interpret the results

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import (  
    accuracy_score,  
    classification_report,  
    confusion_matrix )
```

Loading the Dataset

The Titanic dataset used in Week 2 is loaded for further evaluation and optimization.

```
df = pd.read_csv(r"D:\Machine Learning Fundamentals Course\  
Titanic_Dataset\train.csv")  
  
df["Age"] = df["Age"].fillna(df["Age"].median())  
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])  
df["Cabin"] = df["Cabin"].fillna("Unknown")  
  
df = pd.get_dummies(  
    df,  
    columns=["Sex", "Embarked"],  
    drop_first=True  
)  
  
X = df.drop(  
    ["PassengerId",  
     "Name",  
     "Ticket",  
     "Cabin",  
     "Survived"],  
    axis=1  
)  
  
y = df["Survived"]  
  
X_train, X_test, y_train, y_test = train_test_split(  
    X,  
    y,  
    test_size=0.20,  
    random_state=42  
)
```

Training the Baseline Model

A standard Logistic Regression model is trained without hyperparameter tuning.

```

model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
LogisticRegression(max_iter=1000)
y_pred = model.predict(X_test)

```

Model Evaluation

The model is evaluated using Accuracy, Precision, Recall, and F1-Score.

```

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
Accuracy: 0.8100558659217877
print(classification_report(y_test, y_pred))

```

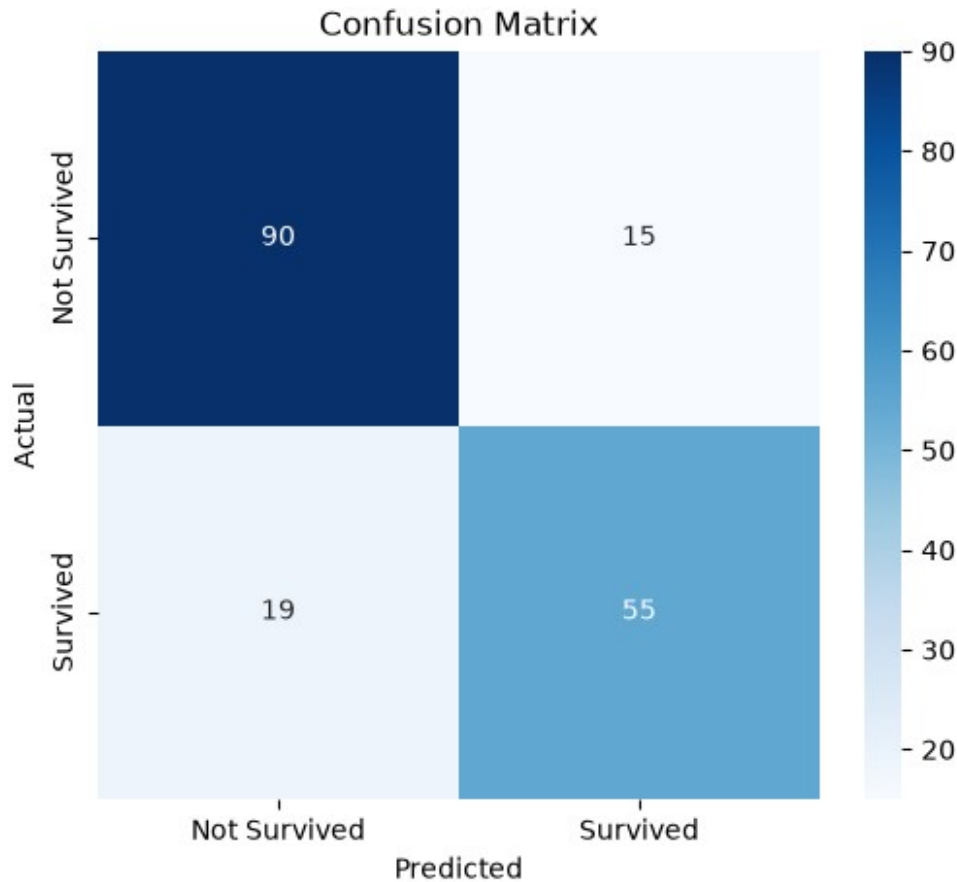
	precision	recall	f1-score	support
0	0.83	0.86	0.84	105
1	0.79	0.74	0.76	74
accuracy			0.81	179
macro avg	0.81	0.80	0.80	179
weighted avg	0.81	0.81	0.81	179

```

cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Not Survived", "Survived"],
    yticklabels=["Not Survived", "Survived"]
)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()

```



Why Accuracy Alone Can Be Misleading

Accuracy measures the percentage of correctly classified samples. However, when one class is much larger than the other, a model can achieve high accuracy simply by predicting the majority class.

For example, if 90% of passengers did not survive, a model that always predicts "Did Not Survive" would achieve 90% accuracy while completely failing to identify survivors.

Precision, Recall, and F1-Score provide a more balanced evaluation because they measure how effectively the model identifies each class rather than only the total number of correct predictions.

Hyperparameter Tuning Using GridSearchCV

GridSearchCV systematically evaluates multiple combinations of model parameters to identify the configuration that produces the best performance.

```

parameters = {
    "C": [0.01, 0.1, 1, 10],
    "solver": ["liblinear", "lbfgs"]
}

grid = GridSearchCV(
    LogisticRegression(max_iter=1000),
    parameters,
    cv=5,
    scoring="accuracy"
)

grid.fit(X_train, y_train)

GridSearchCV(cv=5, estimator=LogisticRegression(max_iter=1000),
             param_grid={'C': [0.01, 0.1, 1, 10],
                         'solver': ['liblinear', 'lbfgs']},
             scoring='accuracy')

print(grid.best_params_)
{'C': 1, 'solver': 'liblinear'}

best_model = grid.best_estimator_
y_pred_tuned = best_model.predict(X_test)

print(classification_report(
    y_test,
    y_pred_tuned
))

```

	precision	recall	f1-score	support
0	0.79	0.85	0.82	105
1	0.76	0.69	0.72	74
accuracy			0.78	179
macro avg	0.78	0.77	0.77	179
weighted avg	0.78	0.78	0.78	179

```

tuned_accuracy = accuracy_score(
    y_test,
    y_pred_tuned
)

print(tuned_accuracy)
0.7821229050279329

```

Performance Comparison

The table below compares the original Logistic Regression model with the tuned model obtained using GridSearchCV.

```
comparison = pd.DataFrame({
    "Model": [
        "Original",
        "Tuned"
    ],
    "Accuracy": [
        accuracy,
        tuned_accuracy
    ]
})
```

comparison

	Model	Accuracy
0	Original	0.810056
1	Tuned	0.782123

Interpretation

The tuned Logistic Regression model was evaluated after optimizing two hyperparameters using GridSearchCV.

The comparison table shows whether hyperparameter tuning improved model performance. Even if the improvement is small, this process demonstrates a systematic approach to model optimization instead of relying on default settings.

Conclusion

In this task, the Logistic Regression model developed in Week 2 was evaluated using Accuracy, Precision, Recall, and F1-Score.

The model was further optimized using GridSearchCV to tune multiple hyperparameters. The tuned model was compared with the original model, demonstrating how systematic optimization can improve predictive performance.

This task highlights the importance of using multiple evaluation metrics and hyperparameter tuning to build reliable machine learning models.